



CS4002 Applied Programming

National University of Computer & Emerging
Sciences
Islamabad, Pakistan

Ms. Umarah Qaseem



1

Content

- Introduction to Object Oriented Programming
- Pillars of OOP
- Understanding basics through a case study
- Hello World of OOP

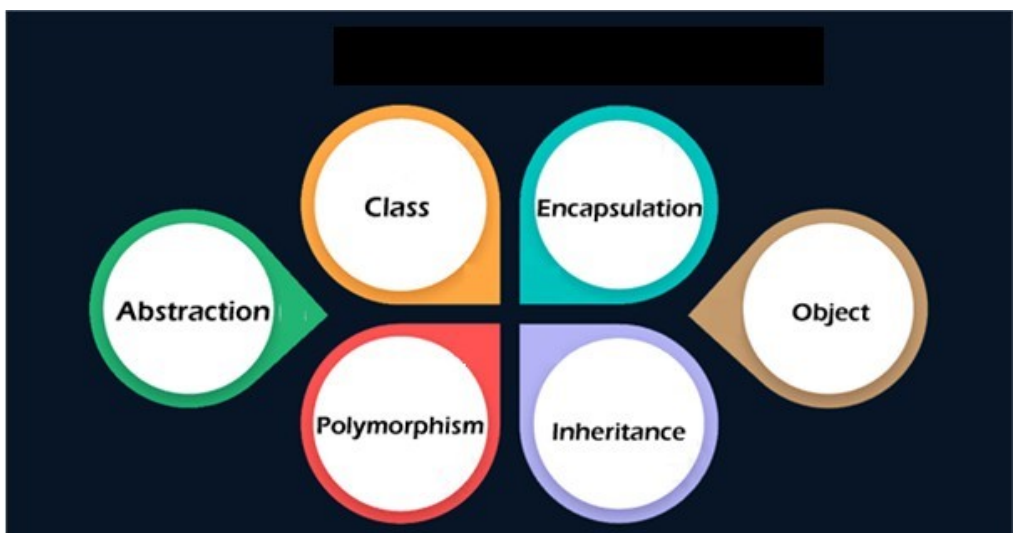
2

Object Oriented Programming

3

3

Basic Concepts of OOP



4

Basic Concepts of **OOP**

1. Object
2. Class
3. Abstraction
4. Inheritance
5. Polymorphism
6. Encapsulation

5

Why Objects?

6

At the end of the day...
computers just manipulate 0's and 1's

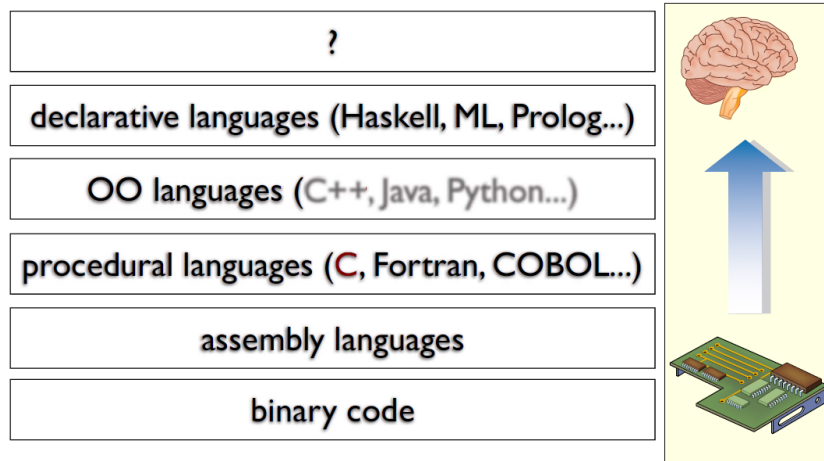


Figure by MIT OpenCourseWare.

But *binary is hard (for humans) to work with...*

7

Towards a higher level of abstraction



6

Figure by MIT OpenCourseWare.

8

There are always trade-offs

High-level languages

- ▶ simpler to write & understand
- ▶ more library support, data structures

Low-level languages

- ▶ closer to hardware
- ▶ more efficient (but also dangerous!)

9

Object-oriented Programming (OOP)

Object-oriented programming approach organizes programs in a way that *mirrors the real world*, in which all **objects** are associated with both *attributes* and *behaviors*

Object-oriented programming involves *thinking in terms of objects*

An OOP program can be viewed as a *collection of cooperating objects*

10

Classes

A class is like a cookie cutter; it defines the shape of objects

Objects are like cookies; they are **instances** of the class



Photograph courtesy of [Guillaume Brialon](#) on Flickr.

11

Classes in OOP

- *Classes are constructs/templates that define objects of the same type.*
- A class uses **variables** to define **data fields**
- **functions** to define **behaviors**.
- Additionally, *a class provides a special type of function, known as **constructors**,*
 - Invoked to **construct objects** from the class.

12

Objects in OOP

- An **object** has a **unique identity, state, and behaviors**.
- The **state** of an **object** consists of *a set of data fields* (also known as **properties**) with their **current values**.
- The **behavior** of an **object** is **defined by a set of functions**

13

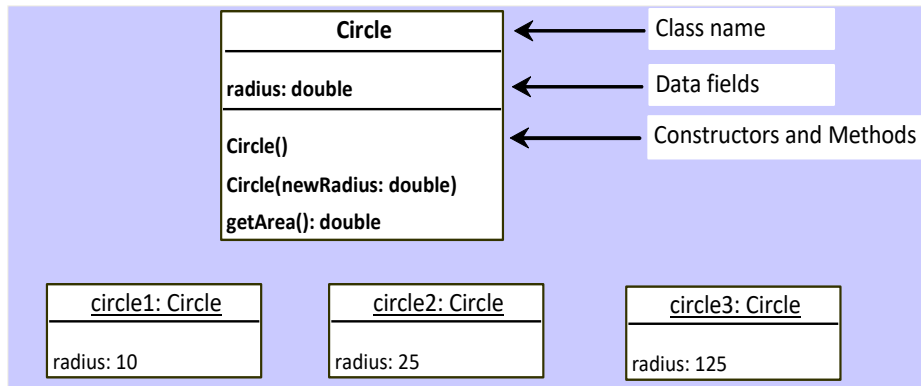
Class and Object

A class is template that defines what an object's data and function will be.

An object is an instance of a class (the terms object and instance are often interchangeable).

14

UML Diagram for Class and Object



15

Class in C++ - Example

```

class foo
{
    private:
        int data;
    public:
        void memfunc (int d)
        { data = d; }
};

```

Annotations for the C++ class example:

- Keyword:** class
- Name of class:** foo
- Braces:** { and }
- Keyword private and colon:** private:
- Private functions and data:** int data;
- Keyword public and colon:** public:
- Public functions and data:** void memfunc (int d) { data = d; }
- Semicolon:** ;

16

Class in C++ - Example

```
class Circle
{
public:
    // The radius of this circle
    double radius;

    // Construct a circle object
    Circle()
    {
        radius = 1;
    }

    // Construct a circle object
    Circle(double newRadius)
    {
        radius = newRadius;
    }

    // Return the area of this circle
    double getArea()
    {
        return radius * radius * 3.14159;
    }
};
```

Data field

Constructors

Function

Note:
the special syntax
for constructor
(no return type!)

17

Class is a Type

You can use **primitive data types** to define variables.

You can also use **class names** to declare object names.

In this sense, a class is an abstract data-type or user-defined data type.

18

Class Data Members and Member Functions

The **data items** within a class are called **data members** or **data fields** or **instance variables**

Member functions are functions that are included within a class. Also known as **instance functions**.



19

Object Creation - Instantiation

In C++, you can **assign a name** when **creating an object**.

A **constructor** is **invoked** when an **object** is created.

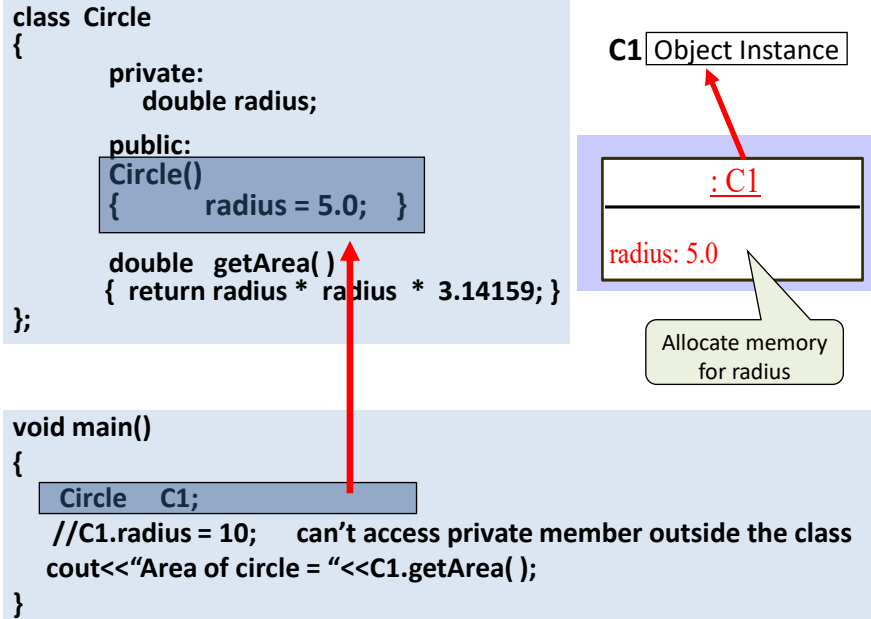
The syntax to create an object using the **no-arg constructor** is

```
ClassName objectName;
```

Defining objects in this way means **creating them**. This is also called **instantiating them**.

20

A Simple Program – *Object* Creation



21

Activity

- ❑ Make a UML diagram for the following scenario.
- ❑ Consider an example named **Saleem**; Saleem is 25 years student old and secured 86 marks in AP test. The Saleem may be represented in a program as an object of class student.
 - The data part of the class would be (name: **Saleem**, age: 25, Marks: 86)
 - The behavioral part of the class may be collection of programs (retrieval of data, Change of Age, Display)

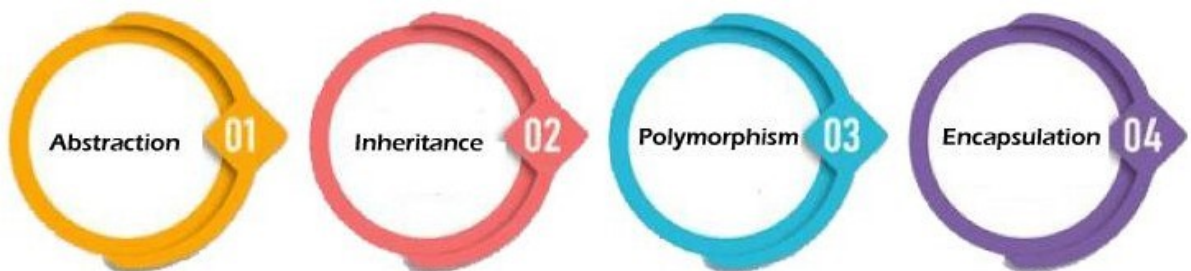
22

Example

Student
DATA Name Date-of-birth Marks
FUNCTIONS Total Average Display

23

Pillars of OOP



24

1 - Abstraction

- Refers to the act of representing essential features without including the background details
- It's about creating a simplified view of an object, exposing only what's necessary.
- Example: When you drive a car, you just need to know how to use the steering wheel, pedals, and gears. You don't need to know how the engine works internally.
- Abstraction is implemented using abstract classes and interfaces in C++.

25

Data Hiding

- Principle of data hiding helps to build secure programs that can't be invaded by code in other parts of the program
- Data hiding is a technique used to implement abstraction.
- Example: The internal variables of a class are often made private to prevent direct access from outside the class.
- In C++, data hiding is achieved using access specifiers (private, protected, public)

26

Abstraction - Example



Driver's View

Engineer's View



27

2 - Inheritance

- Process by which objects of one class acquire the properties of another class
- Provides the idea of reusability
- Additional features can be added to an existing class without modifying it

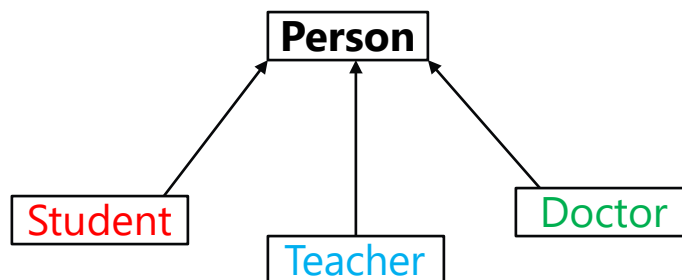
28

Inheritance

- If a class **B** inherits from class **A** then it contains all the characteristics (information structure and behavior) of class **A**
- The parent class is called **base class** and the child class is called **derived class**
- Besides inherited characteristics, **derived** class may have its own unique characteristics

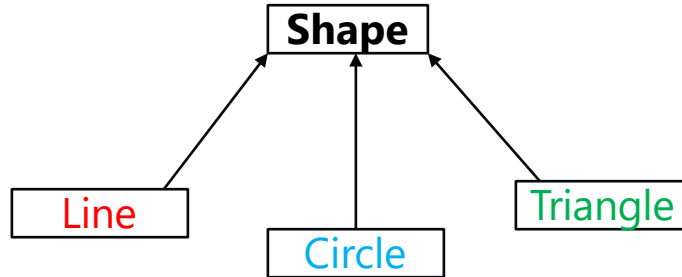
29

Inheritance - Example



30

Inheritance - Example



31

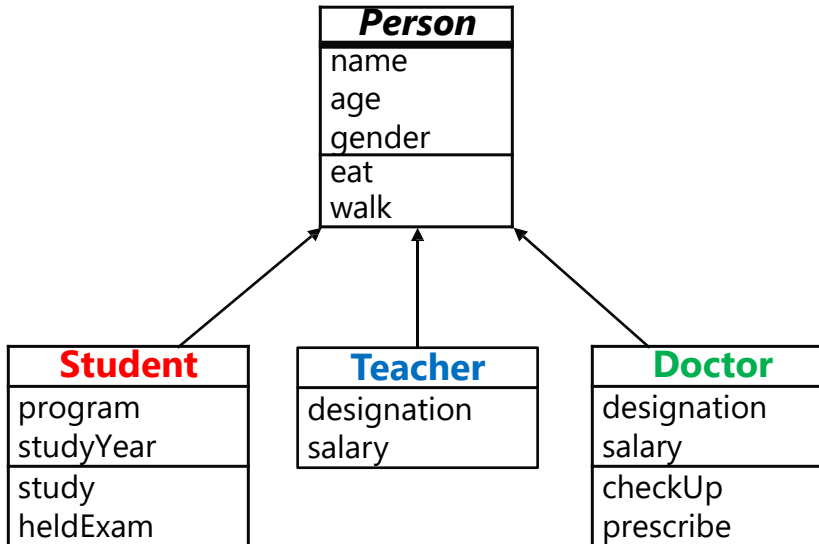
Inheritance

➤ "IS A" or "IS A KIND OF" Relationship

- Each derived class is a special kind of its base class

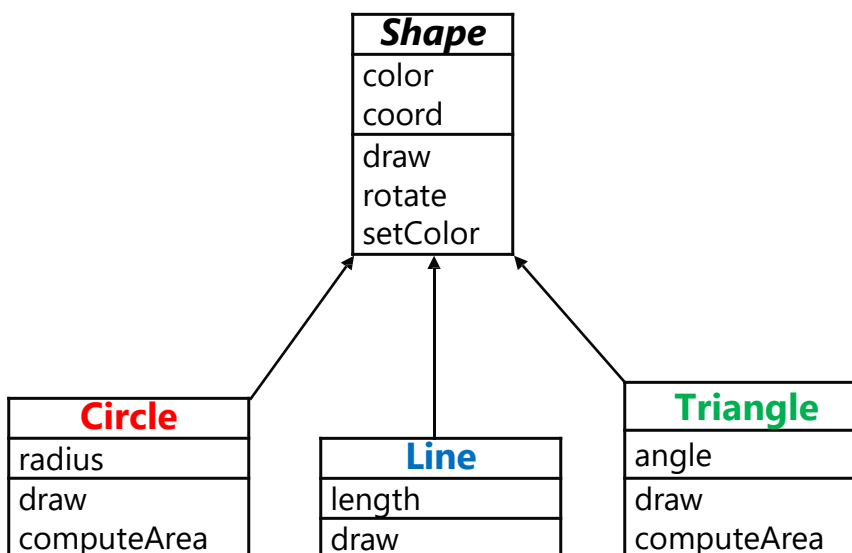
32

"IS A" Relationship - Example



33

"IS A KIND OF" Relationship - Example



34

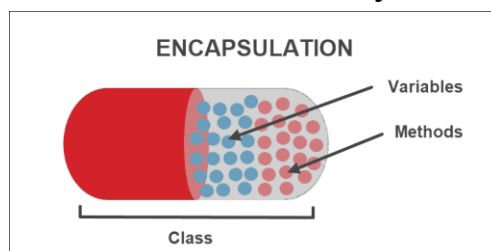
3 - Polymorphism

- Ability to take more than one form
- Feature that allows a function or operator to be given more than one definition
- An operation may exhibit different instance
- Behavior depends upon the type of data used
- Operator overloading and function overloading are examples of Polymorphism

35

4 - Encapsulation

- The wrapping up of data and function into a single unit (called class)
- The data is not accessible to the outside world and only those functions which are wrapped in the class can access it
- Functions provide the interface between the objects data and the program



36

Case Study: Library Management System

➤ Scenario:

- Campus library wants to digitize its management system. They need a software solution to handle books, patrons, borrowing, returns, and library staff management.

37

Case Study: Library Management System

Without OOP:

- Initially, imagine trying to create this system using just functions and data structures, without OOP. You might end up with:
 - Separate arrays or lists for books, patrons, and staff
 - Numerous functions to handle **operations** like adding books, registering patrons, borrowing, returning, etc.
 - Complex **data structures** to represent relationships between books and patrons

This approach would quickly become unwieldy, hard to maintain, and difficult to expand.

38

Case Study: **Library Management System**

With **OOP**:

Now, let's see how OOP can help organize this system more effectively:

1. **Classes and Objects:**

- Book class: Represents individual books
- Patron class: Represents library members
- Staff class: Represents library staff
- Library class: Represents the library itself

39

Case Study: **Library Management System**

2. **Encapsulation:**

- Book class encapsulates data like title, author, ISBN, and status (available/borrowed)
- Patron class encapsulates data like name, ID, borrowed books
- Methods in each class handle their specific operations

40

Case Study: **Library Management System**

3. Inheritance:

- Person class: Base class for common attributes
- Patron and Staff classes inherit from Person

41

Case Study: **Library Management System**

4. Polymorphism:

- Different types of books (TextBook, Novel, Reference) can inherit from Book class
- Each can have its own implementation of methods like calculateLateFee()

42

Case Study: Library Management System

5. Abstraction:

- Library class provides a high-level interface for operations, hiding complex implementations

43

Case Study: Library Management System

Benefits of OOP

1. **Modularity:** Each class represents a distinct entity in the library system
2. **Reusability:** Classes like Person can be reused for both Patron and Staff
3. **Flexibility:** Easy to add new types of books or extend patron functionalities
4. **Maintainability:** Changes to book handling only require modifications to the Book class
5. **Scalability:** The system can easily grow to include more features like reservations, inter-library loans, etc

44